

1 工具篇：快速上手 AI + 开发环境搭建指南

这是一份专为科研新人打造的“从零到一”科研工程化工具与代码基础知识沉淀指南。它将以最直观、通俗的方式，帮助新人快速跨越科研初期的技术壁垒。

1.1 Python 环境快速搭建（零基础友好）

- **Python：科研的核心生产力工具**
- **用处认识：**Python 是目前人工智能、数据分析和科研计算的第一语言。你可以把它理解为一个“万能地基”，通过安装不同的“扩展包”/“宏包”（如 `pandas` 算数据，`openai` 调模型），就能实现各种功能。
- **下载与环境推荐：**
 - **不要**直接去官网下载纯净版 Python（管理多项目时容易环境冲突）。
 - **推荐下载 Miniconda**（轻量级环境管理工具）：[Miniconda 官网](#)。它允许你为不同的科研任务创建隔离的虚拟环境。

1.1.1 . Python 语言核心逻辑

Python 的核心思想是“极其接近人类英语的伪代码”。科研新人必须掌握以下四大最基础语法：

Python

```
# 1. 变量与数据类型
case_title = "门静脉再通" # 字符串 (String)
patient_id = 10025        # 整数 (Integer)

# 2. 导入工具包（关键）
import json # 导入内置的 json 处理工具

# 3. 条件判断（注意：Python 靠“缩进”来区分代码块，通常是4个空格）
if patient_id > 10000:
    print("高级病例记录")
else:
    print("常规病例记录")

# 4. 函数定义与调用
def process_data(input_file):
    # 处理逻辑...
    return "处理成功"
```

1.2 VS Code：地表最强代码编辑器

- **用处认识：**VS Code (Visual Studio Code) 不是编译器，它只是一个“超级记事本”。它的强大之处在于**插件生态**。通过安装插件，它可以写 Python、看 Markdown、改 LaTeX、连服务器，是科研人的“一站式工作台”。
- **下载链接：**[VS Code 官网](#)
- **必备插件推荐**（在左侧正方形积木图标处搜索安装）：

- Python (代码高亮与自动补全)
- Markdown All in One (Markdown 写作增强)
- Chinese (Simplified) (汉化界面)

操作页面介绍: [VS Code 界面介绍哔哩哔哩bilibili](#)

1.3 API 中转大平台 / 大模型 API 调用

- 用处认识:
 - **大模型 API**: 把大模型 (如 GPT-4、DeepSeek) 的核心能力封装成一段接口。你不需要在网页上聊天, 而是用 Python 代码直接批量发送提示词 (Prompt) 并获取结构化结果。
 - **API 中转大平台**: 由于网络限制或多模型切换需求, 中转平台将各类官方 API 整合在一起, 提供统一的访问地址 (BASE_URL) 和统一的账户计费, 极大地方便了国内科研人员。
- **基本调用逻辑**: 通常只需要在代码中配置三个要素:
 1. API_KEY (你的密钥令牌)
 2. BASE_URL (中转平台的接口地址)
 3. MODEL_NAME (具体使用的模型名称, 如 gpt-4o)

1.3.1 国内可用的大模型 API 调用方法

1.3.1.1 (1) 常见平台

- DeepSeek API (推荐, 性价比高)
- 阿里百炼 DashScope
- 智谱 AI GLM

1.3.1.2 (2) 标准调用模板 (Python)

```
import requests

url = "https://api.deepseek.com/v1/chat/completions"
headers = {
    "Authorization": "Bearer YOUR_API_KEY",
    "Content-Type": "application/json"
}

data = {
    "model": "deepseek-chat",
    "messages": [
        {"role": "user", "content": "帮我解释逻辑回归"}
    ]
}

response = requests.post(url, headers=headers, json=data)
print(response.json())
```

1.3.2 ChatGPT / Claude / Gemini 协同使用策略

(1) 分工策略

- ChatGPT: 代码生成 + 结构化表达
- Claude: 长文本润色 + 论文写作
- Gemini: 信息检索 + 多模态理解

(2) 最佳实践

不要“只问一个模型”，要“多模型交叉验证”

1.4 . Markdown 文本工具：文献笔记与报告利器

- **用处认识：**传统的 Word 文档排版容易混乱，且无法优雅地展示代码。Markdown (.md) 是一种轻量级标记语言，让你用纯文本的输入方式，写出排版极其精美的文档、公式和代码块。
- **工具推荐：**

Typora（最推荐的所见即所得编辑器，界面极其优雅）：[Typora 官网](#)

VS Code 配合 Markdown 插件。

1.4.1 . Markdown (MD) 语法速成

MD 的核心是“用符号代替鼠标排版”。

Markdown

```
# 一级标题（注意：# 后面有一个空格）
## 二级标题

这是正文。我们可以加粗**重要科研结论**，或者使用斜体*突出显示*。

### 常用科研排版：
1. **代码块**：用三个反引号包裹
```python
print("Hello, Research!")
```

1. 数学公式（支持 LaTeX 语法）：

- 行内公式：

$$E = mc^2$$

- 独立公式：

$$\mu = \frac{1}{N} \sum_{i=1}^N X_i$$

2. 文献引用引用：

这是一个经典的引用段落。

```
3. GitHub 操作页面与 Git 极简工作流
新手面对 GitHub 复杂的网页往往会迷路，核心关注以下三个核心概念：
```

- \* **Repository** (仓库 / Repo) \*\*: 你在 **GitHub** 上的项目文件夹。
- \* **Commit** (提交) \*\*: 你对代码进行的一次存档记录。
- \* **Clone** (克隆) \*\*: 把 **GitHub** 上的开源项目完整下载到本地电脑上。

#### 🛠️ 常用 **Git** 三板斧命令 (在 **VS Code** 终端运行):

1. ``git add .``: 把当前文件夹下所有修改过的文件移入“暂存区”(准备存档)。
  2. ``git commit -m "优化了大模型提示词"``: 正式存档, ``-m`` 后面写的是你的修改注释。
  3. ``git push origin main``: 把本地的存档同步推送到云端的 **GitHub** 仓库中。
- \* 如果不习惯命令行, **VS Code** 左侧的 **“源代码管理”** 图标 (分叉树形状) 提供了全图形化的点击操作, 非常适合新人。 \*

---

## ## 模块三: 基础常识与 **Bug** 溯源 (科研自救篇)

### ### 1. 常见文件格式 (扩展名) 全解析

科研中你会遇到各种各样的文件, 分清它们的用途至关重要:

格式	全称 / 类型	科研中的典型用途	新人避坑指南
:---	:---	:---	:---
<code>**.txt</code>	<b>Text</b> (纯文本)	记录简单的 <b>Readme</b> 或者是 <b>requirements</b> (依赖包列表)。	没有任何排版样式, 全平台通用。
<code>**.md</code>	<b>Markdown</b>	实验日志、项目说明书、学术笔记。	本质也是纯文本, 用 <b>Typora</b> 或 <b>VS Code</b> 打开效果最好。
<code>**.json</code>	<b>JavaScript Object Notation</b>	结构化数据存储。例如大模型提取出来的患者实体数据。	采用 <code>{ "键": "值" }</code> 的键值对形式, 千万不要漏掉逗号或括号。
<code>**.tex</code>	<b>LaTeX</b> 文档	国际顶级期刊 ( <b>IEEE</b> , <b>Elsevier</b> 等) 的论文排版源码。	需要配合 <b>LaTeX</b> 编译器 (如 <b>Overleaf</b> 或 <b>TexLive</b> ) 渲染成 <b>PDF</b> 。
<code>**.exe</code>	<b>Executable</b> (可执行文件)	<b>windows</b> 系统的软件安装程序或独立运行工具。	运行某些开源科研工具时可能会遇到, <b>Linux</b> 系统无法直接运行。

---

### ### 2. 代码常见报错介绍与 **Bug** 溯源

写代码一定会报错, 报错信息是代码在向你“求救”, 而不是“挑衅”。请引导新人学会看报错的 **“最后一行”**。

#### 🚨 常见的三大基础报错:

1. `**.SyntaxError: invalid syntax` (语法错误) \*\*
  - \* **原因**: 你写错了 **Python** 的基本规矩。比如括号没闭合、少写了冒号 `:`:``、或者不小心混入了中文的标点符号 (如逗号 ``,``、引号 ``,``)。
2. `**.ModuleNotFoundError: No module named '...'` (找不到模块) \*\*
  - \* **原因**: 你尝试 ``import`` 一个你根本没有安装的第三方库。
  - \* **解法**: 去终端运行 ``pip install 库名`` (例如提示找不到 ``openai``, 就运行 ``pip install openai``)。
3. `**.FileNotFoundError: [Errno 2] No such file or directory` (找不到文件) \*\*
  - \* **原因**: 代码要读取的文件路径写错了, 或者文件根本不在那个位置。
  - \* **解法**: 检查路径, 推荐使用 **“绝对路径”**, 或者检查 **VS Code** 当前的工作区目录。

#### 🔍 科学的 **Bug** 溯源四步法:

- \* **第一步: 看末尾**。直接拉到报错信息的最后一行, 看清报错的 **“类型”** 和 **“具体描述”**。
- \* **第二步: 看行号**。报错信息会向上追溯, 找到属于你写的那个 ``.py`` 文件的那一行, 点击链接可以直接跳转到报错代码。
- \* **第三步: 精确复制**。将最后一行的错误信息 (去掉具体的本地路径) 直接复制, 利用搜索引擎或向大模型提问。

\* \*\*第四步：打印检查法（`print`）。\*\* 在报错代码的前一行，用 `print()` 打印出相关的变量，看看它的内容和类型是不是你预期的样子。

---

## ## 🧠 新人进阶学习资源推荐

为了让团队新人能够有系统性地延伸学习，建议将以下优质链接同步沉淀到团队文档中：

\* \*\*Python 零基础快速入门\*\*：[廖雪峰 Python 教程]

(<https://www.liaoxuefeng.com/wiki/1016959663602400>)（公认最适合中文读者入门的经典教程）

\* \*\*Git 与 GitHub 动画通俗教程\*\*：[Learn Git Branching]

([https://learngitbranching.js.org/?locale=zh\\_CN](https://learngitbranching.js.org/?locale=zh_CN))（通过互动游戏和动画彻底搞懂 Git 分支与提交流程，极度推荐）

\* \*\*Markdown 官方语法指南\*\*：[Markdown 官方教程](<https://markdown.com.cn/>)（十分钟即可通读上手的全指南）

\* \*\*LaTeX 论文排版云平台\*\*：[Overleaf 官网](<https://www.overleaf.com/>)（科研新人写论文必备，免去本地配置复杂环境的痛苦）

你可以直接将此结构完整地复制到团队的知识库中（如 Notion、语雀或 GitHub wiki 页面），作为新人的“生存第一课”。对于这些基础设施，新人在搭建或调用时最容易卡在哪个环节？我可以针对性地为你补充更详细的排版。

## 1.5 . Git 与 GitHub：科研成果的“后悔药”与发布会

- 用处认识：
  - **Git (工具)**：版本控制系统。你可以把它理解为“代码摄像机”。每当你代码有重大修改，就“拍个照”（Commit）。如果代码改崩了，可以一键“时空穿梭”回到过去的任意版本。
  - **GitHub (平台)**：全球最大的代码托管网站（代码界的人民广场）。你可以把本地用 Git 记录的代码传到 GitHub 上作为云端备份，或者开源给全世界的审稿人和同行看。
- 下载链接：[Git 官方下载](#) | [GitHub 官方网站](#)

### 1.5.1 . GitHub 操作页面与 Git 极简 workflow

新手面对 GitHub 复杂的网页往往会迷路，核心关注以下三个核心概念：

- **Repository (仓库 / Repo)**：你在 GitHub 上的项目文件夹。
- **Commit (提交)**：你对代码进行的一次存档记录。
- **Clone (克隆)**：把 GitHub 上的开源项目完整下载到本地电脑上。

### 1.5.2 🖥️ 常用 Git 三板斧命令（在 VS Code 终端运行）：

1. `git add .`：把当前文件夹下所有修改过的文件移入“暂存区”（准备存档）。
  2. `git commit -m "优化了大模型提示词"`：正式存档，`-m` 后面写的是你的修改注释。
  3. `git push origin main`：把本地的存档同步推送到云端的 GitHub 仓库中。
- 如果不习惯命令行，VS Code 左侧的“源代码管理”图标（分叉树形状）提供了全图形化的点击操作，非常适合新人。

# 2 实战篇：报错 Debug + Prompt 工程方法论

## 2.1 基础常识与 Bug 溯源（科研自救篇）

### 2.1.1 . 常见文件格式（扩展名）全解析

科研中你会遇到各种各样的文件，分清它们的用途至关重要：

格式	全称 / 类型	科研中的典型用途	新人避坑指南
<code>.txt</code>	Text (纯文本)	记录简单的 Readme 或者是 requirements（依赖包列表）。	没有任何排版样式，全平台通用。
<code>.md</code>	Markdown	实验日志、项目说明书、学术笔记。	本质也是纯文本，用 Typora 或 VS Code 打开效果最好。
<code>.json</code>	JavaScript Object Notation	结构化数据存储。例如大模型提取出来的患者实体数据。	采用 <code>{"键": "值"}</code> 的键值对形式，千万不要漏掉逗号或括号。
<code>.tex</code>	LaTeX 文档	国际顶级期刊（IEEE, Elsevier 等）的论文排版源码。	需要配合 LaTeX 编译器（如 Overleaf 或 TexLive）渲染成 PDF。
<code>.exe</code>	Executable (可执行文件)	Windows 系统的软件安装程序或独立运行工具。	运行某些开源科研工具时可能会遇到，Linux 系统无法直接运行。

### 2.1.2 . 代码常见报错介绍与 Bug 溯源

写代码一定会报错，报错信息是代码在向你“求救”，而不是“挑衅”。请引导新人学会看报错的**最后一行**。

#### 2.1.2.1 🛡️ 常见的三大基础报错：

##### 1. `SyntaxError: invalid syntax`（语法错误）

- 原因：你写错了 Python 的基本规矩。比如括号没闭合、少写了冒号 `:`、或者不小心混入了中文的标点符号（如逗号 `,`、引号 `“”`）。

##### 2. `ModuleNotFoundError: No module named '...'`（找不到模块）

- 原因：你尝试 `import` 一个你根本没有安装的第三方库。
- 解法：去终端运行 `pip install 库名`（例如提示找不到 `openai`，就运行 `pip install openai`）。

##### 3. `FileNotFoundError: [Errno 2] No such file or directory`（找不到文件）

- 原因：代码要读取的文件路径写错了，或者文件根本不在那个位置。
- 解法：检查路径，推荐使用**绝对路径**，或者检查 VS Code 当前的工作区目录。

### 2.1.2.2 🔍 科学的 Bug 溯源四步法：

- **第一步：看末尾。** 直接拉到报错信息的最后一行，看清报错的**类型**和**具体描述**。
- **第二步：看行号。** 报错信息会向上追溯，找到属于你写的那个 `.py` 文件的那一行，点击链接可以直接跳转到报错代码。
- **第三步：精确复制。** 将最后一行的错误信息（去掉具体的本地路径）直接复制，利用搜索引擎或向大模型提问。
- **第四步：打印检查法（`print`）。** 在报错代码的前一行，用 `print()` 打印出相关的变量，看看它的内容和类型是不是你预期的样子。

### 2.1.2.3 ：交给 AI 的标准 Prompt

#### 高效 Debug Prompt 模板（重点）

你是一个资深Python工程师，请帮我debug以下问题：

【错误信息】

xxx

【代码】

xxx

【数据结构说明】

xxx

请你：

1. 定位错误原因
2. 给出修改后的代码
3. 解释为什么错

这是为你定制的进阶篇知识沉淀指南。本模块专注于“AI + 高效科研绘图”，核心解决如何利用 AI 快速生成高质量可发表的图表，并攻克 AI 在处理数据和生图时的“幻觉（Hallucination）”这一业界痛点。

## 3 进阶篇：科研绘图与降低幻觉率

### 3.1 科研绘图工具矩阵与核心优势

AI 时代的科研绘图分为两大流派：**代码生成流（精确、可交互、适合数据图）**与**图像渲染流（具象、适合机制概念图）**。

#### 3.1.1 . Python (代码生成流的核心)

- **优势：**
  - **绝对精确：** 100% 忠实于你的实验数据，绝无像素层面的“幻觉”或多一根手指的荒谬错误。
  - **高度可定制：** 通过 `matplotlib`、`seaborn` 或 `plotly` 库，从坐标轴像素到配色方案（如 Nature/Science 风格）均可代码级微调。
  - **大模型极度友好：** ChatGPT、Gemini 等大模型写 Python 绘图代码的能力远超其直接生图的能力。
- **适用场景：** 折线图、散点图、热力图、统计图表等基于真实数据的图表。



### 3.1.2 Gemini & Canvas / 专属多模态工具

- 优势：
  - **多模态直观交互**：你可以直接框选图表的某一部分，或者在独立的工作区（如 Canvas 模式）中让 Gemini 针对特定区域进行代码修改或版面调整，无需频繁在聊天框和代码之间切换。
  - **原生代码执行**：Gemini 具备强大的内置代码执行环境（Code Execution），可以先用 Python 计算、验证数据，再将其渲染为可视化图表，极大降低了由于模型逻辑推理错误带来的幻觉。
- 适用场景：需要边看图、边反复微调局部细节的数据分析与绘图任务。

### 3.1.3 . ChatGPT Plus (GPT-4o & Advanced Data Analysis)

- 优势：
  - **沙盒运行环境**：内置强大的高级数据分析功能（DALL-E 3 与 Python 沙盒环境）。你可以直接上传 `.csv` 或 `.xlsx` 原始数据，它会自动编写代码并在后台运行，直接输出高分辨率的图表。
  - **DALL-E 3 的语义理解**：如果是生成机制示意图，DALL-E 3 对科研术语的文本理解力极强，能较好地抽象概念具象化。
- 适用场景：原始数据批量清洗加直接出图、科研机理示意图（概念图）的草稿生成。

### 3.1.4 . 其他专业科研绘图 AI 辅助工具

- Midjourney / Stable Diffusion（艺术级渲染）：
  - 优势：光影和细节极其震撼。适合用于绘制高水平的期刊封面图（Cover Image）或医学、材料学的宏观/微观结构示意图。
- BioRender（结合 AI 的半自动组装工具）：
  - 优势：生物学界的“Canvas”。虽然不是纯 AI 生图，但它现在引入了 AI 辅助布局和图标推荐。提供海量标准化的细胞、器官、实验仪器图标，**绝无幻觉，极度严谨**，是发表医学论文的首选。

## 3.2 高频科研图表类型与应用场景

科研新手往往不知道什么数据该用什么图。以下是顶刊中最常用的高级图表：

📊 科研高频图表速查：		
— 差异表达分析 —	火山图 (Volcano Plot)	-> 筛选显著基因/蛋白
— 密集数据矩阵 —	热力图 (Heatmap)	-> 展现聚类与相关性
— 分布与密度 —	小提琴图 (Violin Plot)	-> 比箱线图多展示了“数据密度”
— 多维特征对比 —	雷达图 (Radar Chart)	-> 评估综合性能/多指标

#### 3.2.1 1. 火山图 (Volcano Plot)

- **特点**：横轴表示变化倍数 ( $\log_2 \text{Fold Change}$ )，纵轴表示显著性 ( $-\log_{10} P \text{ value}$ )。
- **应用**：基因组学、转录组学或多指标筛选。一眼看出哪些因子（左上和右上）具有显著的统计学差异。



### 3.2.2 2. 热力图 (Heatmap)

- **特点：**用颜色的深浅来代表数值的大小，通常结合“层级聚类树”使用。
- **应用：**展示大批量样本之间的相关性、基因表达谱、或者时间序列的特征变化趋势。

### 3.2.3 3. 小提琴图 (Violin Plot)

- **特点：**结合了“箱线图 (Box Plot)”和“核密度图 (Kernel Density Plot)”。
- **应用：**不仅能展示数据的中位数、四分位数，还能优雅地展现出数据在不同区间的**分布密度**（例如判断数据是否呈现双峰分布）。

### 3.2.4 4. 主成分分析图 (PCA Plot)

- **特点：**将成千上万维的数据降维到 2D 或 3D 空间进行散点展示。
- **应用：**用于观察实验组和对照组的样本是否能完美“分离开”，验证实验分类的效果。

## 3.3 如何降伏 AI 绘图的“幻觉”？（核心自救方案）

AI 的幻觉在绘图时的表现：**数据点无中生有、坐标轴刻度错乱、专业英文单词拼错、机制图里的细胞长出奇怪的触手。**

要降低幻觉率，必须从 **System Prompt（系统层规则约束）** 和 **User Prompt（用户层结构输入）** 联合封锁。

### 3.3.1 . 核心防御策略：不要让 AI “直接画”，要让 AI “写代码画”

**金科玉律：**处理真实实验数据时，严禁使用 AI 的图像生成功能（如 DALL-E、Midjourney），必须强制 AI 输出 **Python** 格式

### 3.3.2 System Prompt 模板（为 AI 植入“严谨科学家”的人设）

如果你使用的是自定义 GPTs，或者在开始对话前配置系统提示词，请直接复制以下模板：

⚙️ **System Prompt 模板：**

Plaintext

```
Role
你是一位在 Nature 和 Science 发表过多篇论文的资深科研数据可视化专家与统计学家。

Execution Rules
1. 【严禁幻觉】你处理数据图表的唯一合法方式是生成 Python 代码。严禁在没有运行代码的情况下直接对数据趋势进行主观臆测。
2. 【数据忠实度】生成的图表必须严格匹配用户提供的原始数据，不得对缺失值进行无明示的自动填充，不得擅自平滑曲线。
3. 【学术规范】所有生成的图表必须具备：清晰的坐标轴标签（包含单位）、图例、清晰的刻度。默认使用学术界通用配色（如 Nature 风格的色彩配比，避免高饱和度荧光色）。
4. 【报错自检】如果你的代码在沙盒中运行报错，你必须先自动解析报错并修正代码，确认出图成功后再向用户汇报。
```

### 3.3.3 . User Prompt 模板与框架（结构化提问法）

当你要给 AI 发送具体绘图指令时，使用**结构化 Prompt**，不给 AI 瞎编的空间。

3.3.3.1 📄 场景 A：数据统计图（以小提琴图为例）

User Prompt 模板:

Plaintext

```
> 【任务】请帮我使用 Python 的 Seaborn 库绘制一张高水平学术论文中使用的小提琴图。
> 【数据输入】数据见下方的三明治格式/或已上传的文件：
> - 自变量(X轴)：治疗组别 (Control, Drug_A, Drug_B)
> - 因变量(Y轴)：肿瘤体积缩减率 (Tumor_Reduction_Rate, 单位：%)
> 【视觉规范】
> 1. 配色：请使用优雅的柔和学术色系（例如莫兰迪色或 Nature 风格），拒绝高饱和度颜色。
> 2. 细节：请在小提琴图内部叠加显示“箱线图(Boxplot)”，以便同时观察中位数和四分位间距。
> 3. 字体：全图使用 Arial 字体，标题 14pt，坐标轴标签 12pt。
> 【防幻觉约束】请直接给出完整的、复制即用的 Python 代码，并在代码中包含模拟的示例数据 (DataFrame)，确保我运行代码就能立刻看到效果，不接受伪代码。
> ```

📄 场景 B：科学机制示意图（大模型生图，如概念插图）
大模型（如 DALL-E 3）直接生图极易出现文字拼写幻觉。
> **User Prompt 模板：**
> ```text
> 【物理/生物机理生图任务】
> 【核心主体】绘制一个纳米颗粒 (Nanoparticle) 穿过细胞膜 (Cell Membrane) 进入细胞内部的微观过程示意图。
> 【构图与风格】
> - 风格：3D 科学渲染风格 (Scientific 3D rendering)，具有高清晰度、干净的白色背景。类似 Nature 论文插图。
> - 视角：侧面剖面视角 (Cross-section view)，清晰展示双层磷脂分子结构。
> 【防幻觉核心约束（极重要）】
> 1. 严禁在图片中添加任何由 AI 自动生成的英文文本标签或箭头。我需要一张【纯净无文字】的底图，后续我会自己用 PPT 或 Illustrator 标注，防止文字拼写错误和排版混乱。
> 2. 保持生物学结构的科学性，纳米颗粒呈规则球体，细胞膜双层结构紧密，不要出现不符合常理的扭曲。
> ```

```